



# UNIVERSITY OF BARISHAL

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

---

## **Student Academic Performance Prediction Using Machine Learning Techniques**

**Course:** Artificial Intelligence Lab

**Course Code:** 3206

### **Submitted By**

Shaida Khanom Sharna

Roll : 22 CSE 019

Department of Computer Science and Engineering

University of Barishal

### **Submitted To**

Dr. Tania Islam

Assistant Professor

Department of Computer Science and Engineering

University of Barishal

***Date of Submission: 08-07-2026***

## **1. Introduction**

Academic performance is one of the most closely monitored outcomes in any educational institution, as it directly reflects the effectiveness of teaching methods, student engagement, and the broader learning environment. Traditionally, the identification of at-risk students has relied on the manual judgement of teachers and academic administrators, a process that is time-consuming, subjective, and difficult to scale across large student populations. The emergence of Artificial Intelligence (AI) and Machine Learning (ML) offers a data-driven alternative that can process large volumes of student information and uncover patterns that are not immediately obvious to a human observer.

This report documents the design, implementation, and evaluation of a machine learning system developed to predict students' academic performance category (referred to as "GradeClass") using a structured dataset containing demographic, behavioural, and academic attributes. The project was completed as part of the Final Examination for the course Artificial Intelligence, in which the use of AI-assisted development tools such as ChatGPT, Claude, Gemini, and GitHub Copilot was explicitly permitted, provided that every significant design decision was understood, justified, and explained by the student.

The remainder of this report is organised to mirror the nine tasks specified in the examination brief. Section 2 describes the dataset in detail. Sections 3 through 11 correspond to Task 1 through Task 9 respectively, covering dataset understanding, exploratory data analysis, data cleaning and preprocessing, feature engineering, machine learning model development, model evaluation, model interpretation, a reflection on the use of AI tools during development, and a critical discussion of the system's limitations and ethical considerations. The report concludes with a summary of key findings, a list of references, and supporting screenshot placeholders corresponding to the visual outputs produced in the accompanying Google Colab notebook.

All code, analyses, and results presented in this report were generated in a Google Colab notebook using the Python programming language, together with the pandas, NumPy, scikit-learn, Matplotlib, and Seaborn libraries. Every reported figure, table, and metric in this report is drawn directly from the executed notebook; no results have been fabricated or estimated.

## 2. Dataset Description

The dataset used in this project is the Student Performance Dataset, supplied as a CSV file named `Student_performance_data_.csv`. It contains 2,392 student records described by 15 columns, comprising demographic attributes (Age, Gender, Ethnicity), family-background attributes (ParentalEducation, ParentalSupport), academic-behaviour attributes (StudyTimeWeekly, Absences, Tutoring), extracurricular-engagement attributes (Extracurricular, Sports, Music, Volunteering), and academic-outcome attributes (GPA, GradeClass). A full column-by-column description, exactly as loaded into the notebook, is presented in Table 1.

**Table 1: Description of dataset columns**

| Column            | Data Type                | Description                                                                                      |
|-------------------|--------------------------|--------------------------------------------------------------------------------------------------|
| StudentID         | Integer                  | Unique identifier assigned to each student record (range 1001–3392). Not used for prediction.    |
| Age               | Integer                  | Student's age in years (observed range: 15–18).                                                  |
| Gender            | Integer (coded)          | Student's gender, encoded numerically (0 / 1).                                                   |
| Ethnicity         | Integer (coded)          | Student's ethnic group, encoded numerically (0–3).                                               |
| ParentalEducation | Integer (coded, ordinal) | Highest education level attained by the student's parent(s), encoded numerically (0–4).          |
| StudyTimeWeekly   | Float                    | Average number of hours the student studies per week (approx. 0–20).                             |
| Absences          | Integer                  | Number of school absences recorded for the student (range 0–29).                                 |
| Tutoring          | Integer (binary)         | Whether the student receives tutoring (0 = No, 1 = Yes).                                         |
| ParentalSupport   | Integer (coded, ordinal) | Level of parental support for the student's education, encoded numerically (0–4).                |
| Extracurricular   | Integer (binary)         | Whether the student participates in extracurricular activities (0 = No, 1 = Yes).                |
| Sports            | Integer (binary)         | Whether the student participates in sports (0 = No, 1 = Yes).                                    |
| Music             | Integer (binary)         | Whether the student participates in music-related activities (0 = No, 1 = Yes).                  |
| Volunteering      | Integer (binary)         | Whether the student participates in volunteering activities (0 = No, 1 = Yes).                   |
| GPA               | Float                    | Student's Grade Point Average (observed range: 0.0–4.0).                                         |
| GradeClass        | Float (target)           | Categorical performance class derived from GPA (values 0.0–4.0). Target variable for prediction. |

### 2.1 Target Variable

The target variable for this project is GradeClass, a numerically encoded categorical variable representing a student's overall academic performance band, derived from GPA. Because GradeClass takes a small number of discrete values (0.0 through 4.0) rather than a continuous range, the underlying prediction problem is a multi-class classification problem rather than a regression problem. This classification is examined in detail in Task 1 (Section 3).

## ***2.2 Note on Categorical Encoding***

Several columns in the dataset (Gender, Ethnicity, ParentalEducation, ParentalSupport, Tutoring, Extracurricular, Sports, Music, Volunteering, and GradeClass) are stored as integer codes rather than descriptive text labels. The notebook does not decode these numeric codes into their original category names, and, in keeping with the principle that the notebook is the primary source of truth for this report, no such decoding is asserted here as fact. All analysis, visualisation, and modelling in this report therefore treat these fields in their original numerically encoded form, exactly as they appear in the source file.

## ***2.3 Dataset Origin and Suitability***

The dataset is well suited to the objectives of this examination because it combines numerical, ordinal, and binary features with a well-defined multi-class target, contains no missing values or duplicate records (as verified in Section 4), and is of a size (2,392 rows) that is large enough to support a reliable train–test split while remaining computationally lightweight for a Google Colab environment.

### 3. Task 1: Dataset Understanding

The purpose of this task is to establish a clear understanding of the dataset before any analysis or modelling is performed. This includes identifying the target variable, determining the nature of the prediction problem, separating numerical from categorical features, and reflecting on the practical challenges and opportunities that AI techniques can address.

#### 3.1 Importing Libraries and Loading the Dataset

The notebook begins by importing the core Python libraries required for the project: pandas and NumPy for data manipulation, and Matplotlib and Seaborn for visualisation. The display option `pd.set_option('display.max_columns', None)` is set so that all 15 columns of the dataset are visible when previewing data frames, which is helpful given the relatively wide structure of the dataset.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
pd.set_option('display.max_columns', None)
```

The dataset is then loaded from the CSV file using `pd.read_csv("/content/Student_performance_data_.csv")`, and the first five rows are displayed using `df.head()` to provide an initial view of the data structure. This step is essential because it confirms that the file was read correctly, that column names and data types appear as expected, and that no obvious parsing errors occurred during import.

#### 3.2 Dataset Shape and Structure

The shape of the dataset is examined using `df.shape`, which returns 2,392 rows and 15 columns. The full list of column names is retrieved with `df.columns`, and the structural summary produced by `df.info()` confirms that all 15 columns are fully populated (2,392 non-null entries each) and that the data types consist of 12 integer (int64) columns and 3 floating-point (float64) columns: StudyTimeWeekly, GPA, and GradeClass. Both `df.head()` and `df.tail()` are used to inspect the beginning and end of the dataset, which is good practice for confirming that the file has been loaded in its entirety and that there are no formatting anomalies near the end of the file.

#### 3.3 Target Variable Identification

The target variable is explicitly identified in the notebook as GradeClass, assigned to the variable `target = "GradeClass"`. GradeClass represents a discretised academic performance band derived from each student's GPA.

#### 3.4 Classification or Regression?

Although GradeClass is stored using a floating-point data type (float64), its values are restricted to a small, fixed set of discrete levels (0.0, 1.0, 2.0, 3.0, and 4.0), each corresponding to a distinct performance band rather than a continuous quantity. Consequently, the prediction task addressed throughout this project is a supervised multi-class classification problem, not a regression problem. This distinction is important because it determines which family of machine learning algorithms, evaluation metrics, and preprocessing choices are appropriate. Regression

metrics such as mean squared error would be inappropriate here; instead, classification metrics such as accuracy, precision, recall, and F1-score — used in Task 6 — are the correct choice.

### ***3.5 Numerical and Categorical Features***

Using `df.select_dtypes(include=['int64','float64'])`, all 15 columns are identified as numerical in terms of their stored data type. A separate check using `df.select_dtypes(include=['object'])` returns an empty index, confirming that the dataset contains no columns stored as text (object) type. It is important to note, however, that several of the numerically stored columns are conceptually categorical rather than truly continuous: Gender, Ethnicity, ParentalEducation, Tutoring, ParentalSupport, Extracurricular, Sports, Music, and Volunteering all take a small number of discrete integer values and represent categorical or ordinal information encoded numerically, while Age, StudyTimeWeekly, Absences, and GPA are genuinely continuous or count-based numerical measurements. Recognising this distinction, even though it is not reflected in the raw pandas dtype, is an important part of dataset understanding because it affects later decisions about scaling and encoding.

### ***3.6 Statistical Summary***

The `df.describe()` method is used to generate descriptive statistics (count, mean, standard deviation, minimum, quartiles, and maximum) for every column. Key observations from this summary include: Age ranges from 15 to 18 years with a mean of approximately 16.5; StudyTimeWeekly ranges from close to 0 to approximately 20 hours per week with a mean of about 9.8 hours; Absences range from 0 to 29 with a mean of approximately 14.5; GPA ranges from 0.0 to 4.0 with a mean of approximately 1.91; and GradeClass has a mean of approximately 2.98, suggesting that, on this 0–4 scale, the class distribution is skewed toward the higher-numbered (lower-performing) bands. This initial statistical profile provides a useful baseline against which the results of the more detailed Exploratory Data Analysis in Task 2 can be compared.

### ***3.7 Important Features***

Based on the dataset understanding performed so far and confirmed later through correlation analysis and feature importance ranking (Sections 4 and 8), the features most likely to be important predictors of GradeClass are GPA (by construction, since GradeClass is derived from GPA), Absences (attendance is a well-established predictor of academic outcomes), and StudyTimeWeekly (study habits directly influence performance). Secondary features such as ParentalSupport, Tutoring, and ParentalEducation are also plausible contributors, as the education literature consistently links family support structures to student outcomes.

### ***3.8 Challenges***

Two notable challenges were identified while working with this dataset. First, several features are ordinal or categorical but are encoded as plain integers without accompanying label documentation in the notebook, which creates a risk of misinterpreting the numerical scale (for example, treating an ordinal ParentalSupport level as though it were a continuous quantity with equal spacing between categories). Second, because GradeClass is mathematically derived from GPA, and GPA is included as an input feature, there is an inherent risk of data leakage and artificially inflated model performance, since the model can partly “shortcut” the classification task

by learning the GPA-to-GradeClass mapping rather than learning genuine predictive relationships from the other behavioural and demographic features. This issue is discussed further in Section 12 (Task 9: Critical Discussion).

### 3.9 How AI Can Assist

Artificial Intelligence can assist HealthPlus-style organisations and, in this case, educational institutions in several ways. Machine learning models can automatically flag students who are at risk of poor academic performance early in a term, allowing teachers and academic advisors to intervene proactively rather than reactively. AI-assisted exploratory analysis (as demonstrated throughout this project) also accelerates the process of understanding large datasets, generating visualisations, and identifying the most influential factors behind an outcome, which would otherwise require substantially more manual effort from an analyst.

|   | StudentID | Age | Gender | Ethnicity | ParentalEducation | StudyTimeWeekly | Absences | Tutoring | ParentalSupport | Extracurricular | Sports | Music |
|---|-----------|-----|--------|-----------|-------------------|-----------------|----------|----------|-----------------|-----------------|--------|-------|
| 0 | 1001      | 17  | 1      | 0         | 2                 | 19.833723       | 7        | 1        | 2               | 0               | 0      | 1     |
| 1 | 1002      | 18  | 0      | 0         | 1                 | 15.408756       | 0        | 0        | 1               | 0               | 0      | 0     |
| 2 | 1003      | 15  | 0      | 2         | 3                 | 4.210570        | 26       | 0        | 2               | 0               | 0      | 0     |
| 3 | 1004      | 17  | 1      | 0         | 3                 | 10.028829       | 14       | 0        | 3               | 1               | 0      | 0     |
| 4 | 1005      | 17  | 1      | 0         | 2                 | 4.672495        | 17       | 1        | 3               | 0               | 0      | 0     |

**Figure 1: Notebook Header and Initial Dataset Preview**

## 4. Task 2: Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is performed to build a deeper, evidence-based understanding of the dataset before any modelling is attempted. In the notebook, this task begins with a markdown cell stating the objectives of EDA: understanding the dataset structure, identifying missing values, detecting duplicate records, analysing data distributions, exploring relationships between variables, and visualising important patterns. Each of these objectives is addressed systematically in the cells that follow.

### 4.1 Dataset Structure, Data Types, and Completeness

The dataset shape is re-confirmed as 2,392 rows and 15 columns using `df.shape[0]` and `df.shape[1]`, and the data types of every column are re-examined with `df.dtypes` and `df.info()`, reproducing the same structure established in Task 1. Missing values are then checked column-by-column using `df.isnull().sum()`, which returns zero missing values for every one of the 15 columns, and the aggregate check `df.isnull().sum().sum()` confirms a grand total of zero missing values across the entire dataset. Duplicate rows are checked using `df.duplicated().sum()`, which also returns zero, confirming that every one of the 2,392 records is unique. These results are important because they establish that this particular dataset does not require missing-value imputation or duplicate removal, which simplifies the preprocessing decisions made in Task 3.

## 4.2 Statistical Summary

The `df.describe()` statistical summary is regenerated at this stage, reproducing the same descriptive statistics discussed in Section 3.6 (Age, StudyTimeWeekly, Absences, GPA, and GradeClass ranges and central tendencies). Repeating this step within the EDA task reinforces the numerical profile of the dataset immediately before visual analysis begins.

## 4.3 Visual Analysis

Eight visualisations were produced in the notebook using Matplotlib and Seaborn, covering the class distribution of the target variable, inter-feature correlation, the distribution of the two most important continuous variables (GPA and Age), a categorical breakdown of Gender, two bivariate relationships against GPA, and an outlier-focused boxplot. Each is discussed below in the order it appears in the notebook, together with its corresponding figure placeholder.

### 4.3.1 Distribution of Grade Classes

A count plot of the GradeClass column was generated using `sns.countplot(data=df, x="GradeClass", palette="Set2")` to visualise how many students fall into each of the five performance bands. Understanding the balance (or imbalance) of the target classes is essential before model building, because a heavily imbalanced target can bias a classifier toward the majority class and can make accuracy alone a misleading evaluation metric.

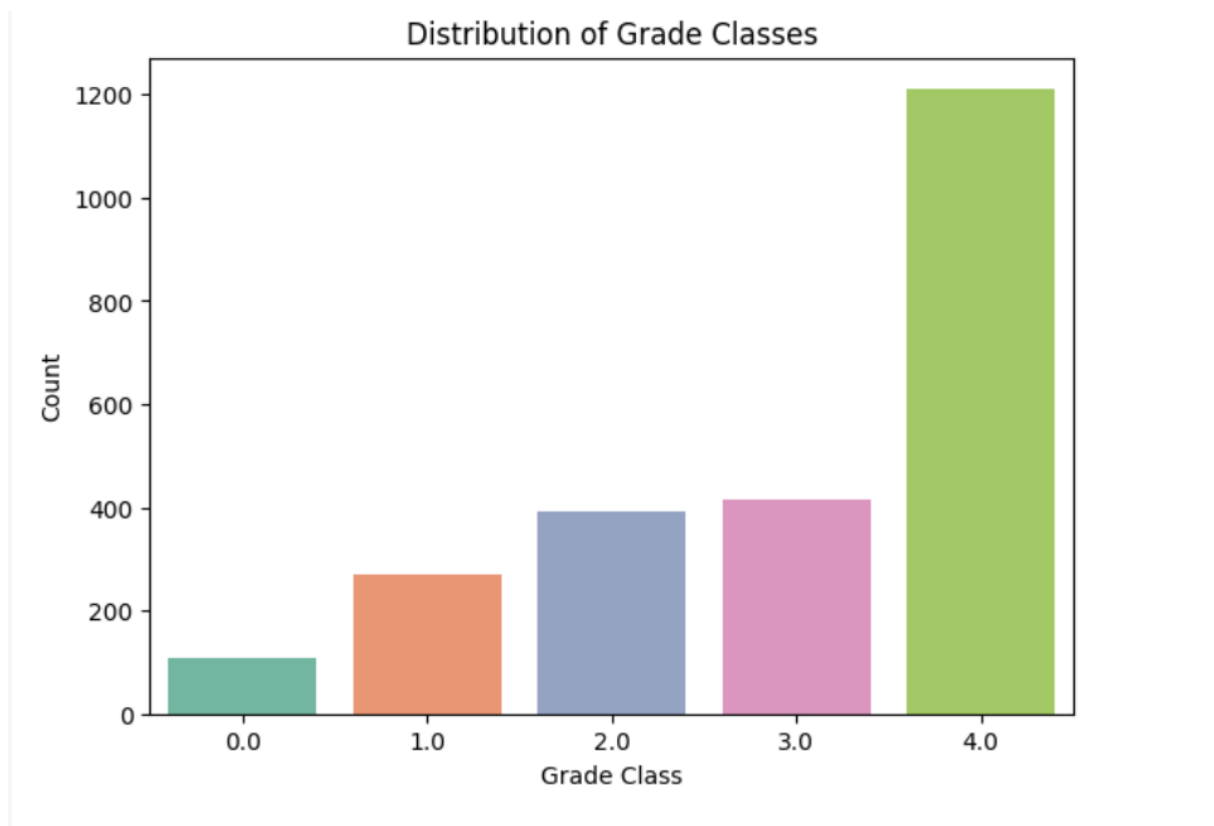


Figure 2: Distribution of Grade Classes



### 4.3.2 Correlation Matrix

A correlation heatmap of all numerical columns was generated using `sns.heatmap(df.corr(), annot=True, cmap="coolwarm")` to visualise the pairwise linear relationships between every pair of features, including the target variable. This step is critical for identifying which features are most strongly associated with GradeClass and for detecting potential multicollinearity between predictor variables before feature engineering and model training are carried out.

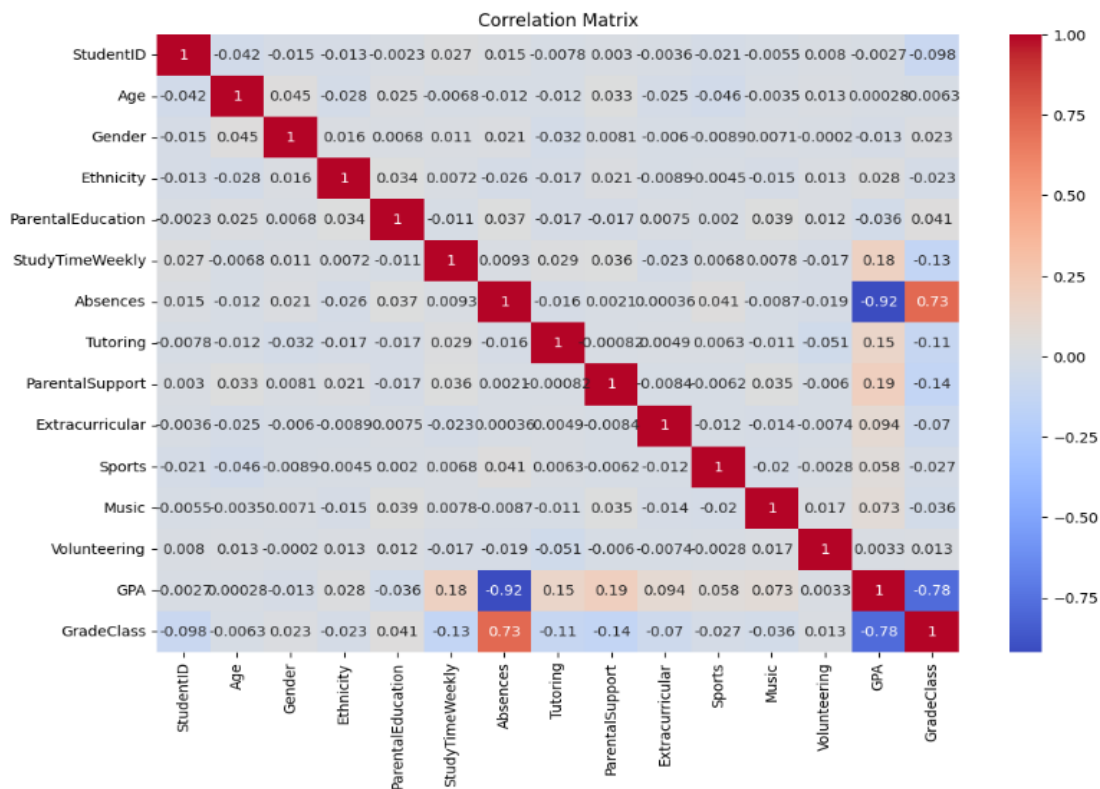
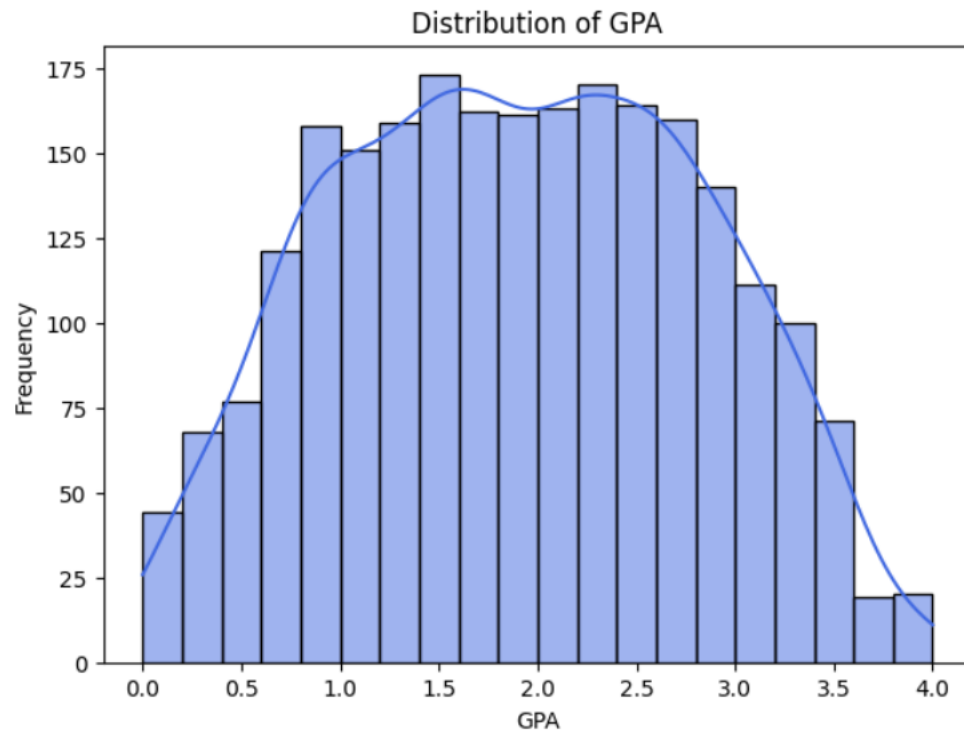


Figure 3: Correlation Matrix Heatmap of All Numerical Features

### 4.3.3 Distribution of GPA

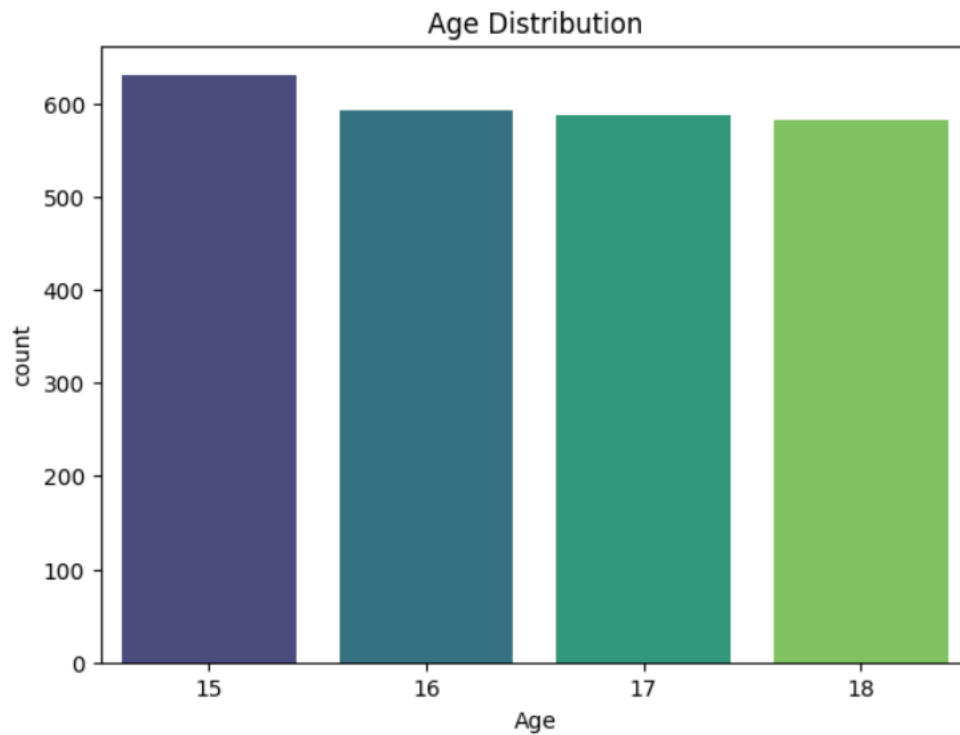
A histogram with an overlaid kernel density estimate was created using `sns.histplot(df["GPA"], bins=20, kde=True, color="royalblue")` to examine the shape of the GPA distribution across the student population. Understanding whether GPA is approximately symmetric, skewed, or multi-modal helps to explain the corresponding shape of the GradeClass distribution, since GradeClass is derived directly from GPA.



**Figure 4: Distribution of GPA**

#### ***4.3.4 Age Distribution***

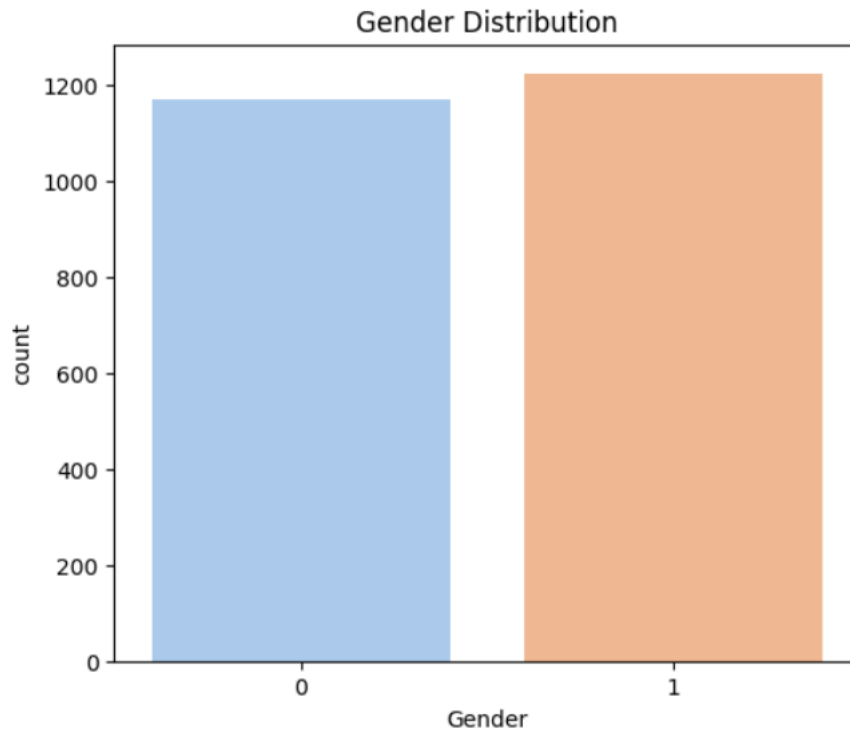
A count plot of the Age column was produced using `sns.countplot(data=df, x="Age", palette="viridis")` to examine how students are distributed across the observed age range of 15 to 18 years. This provides context on the demographic composition of the sample and helps assess whether age is likely to be a meaningful predictor or largely uniform across the dataset.



**Figure 5: Age Distribution of Students**

#### ***4.3.5 Gender Distribution***

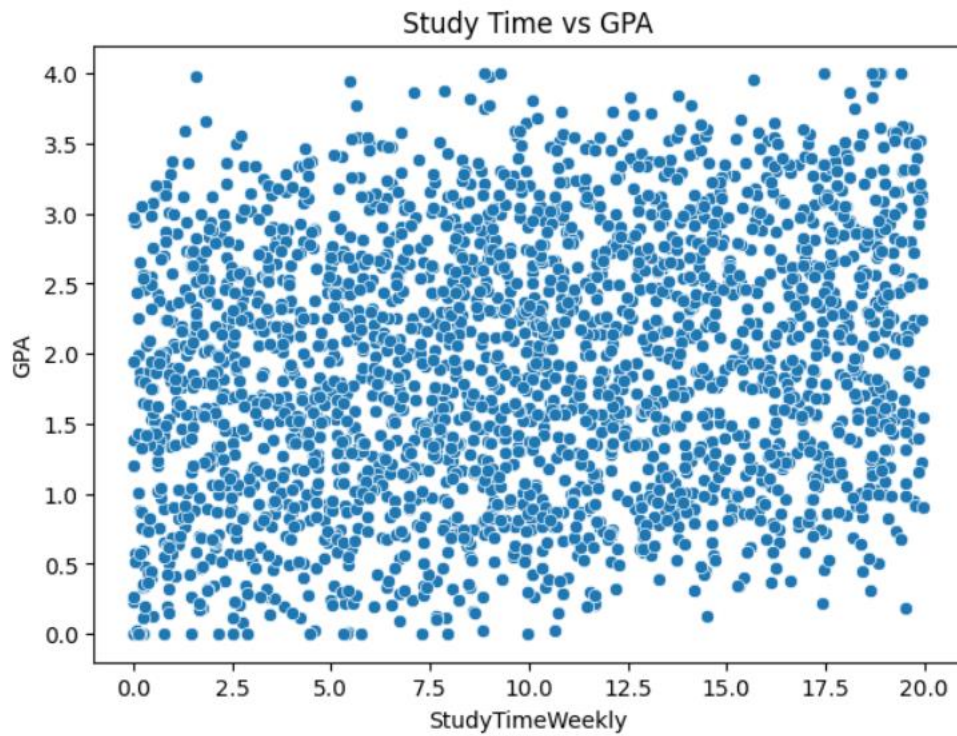
A count plot of the Gender column was produced using `sns.countplot(data=df, x="Gender", palette="pastel")` to examine the balance between the two encoded gender categories in the dataset. A roughly balanced gender split, if present, would suggest that gender is unlikely to introduce a significant class-imbalance bias into the modelling stage.



**Figure 6: Gender Distribution of Students**

#### ***4.3.6 Study Time versus GPA***

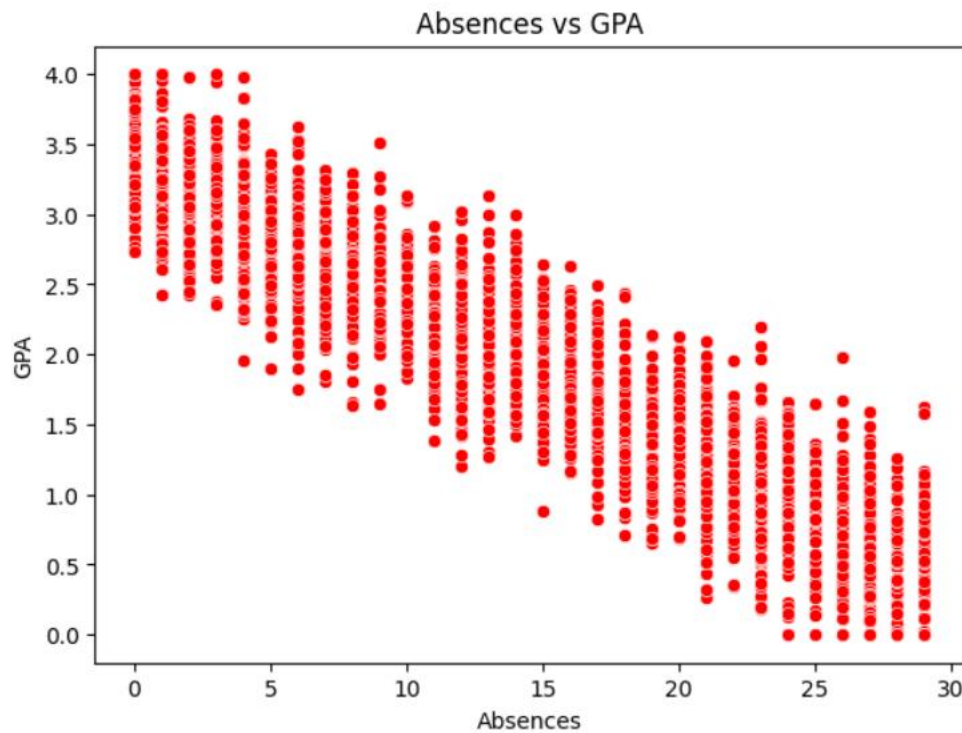
A scatter plot of StudyTimeWeekly against GPA was created using `sns.scatterplot(data=df, x="StudyTimeWeekly", y="GPA")` to visually assess whether students who report studying more hours per week tend to achieve higher GPA scores. This bivariate visualisation complements the correlation matrix by showing the actual shape of the relationship, rather than only its linear strength.



**Figure 7: Scatter Plot of Weekly Study Time versus GPA**

#### ***4.3.7 Absences versus GPA***

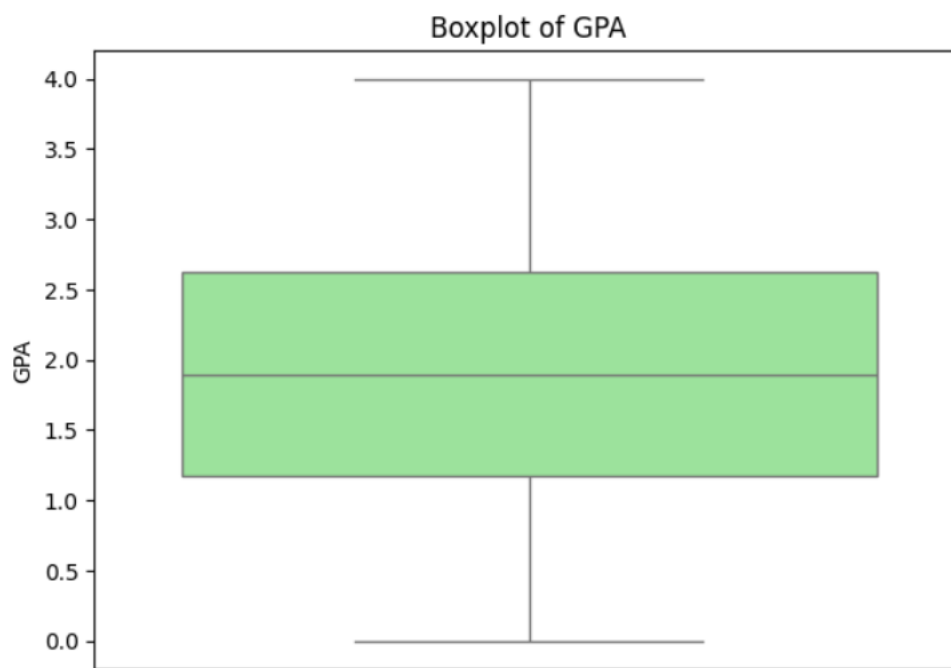
A second scatter plot, this time of Absences against GPA, was created using `sns.scatterplot(data=df, x="Absences", y="GPA", color="red")` to examine whether higher numbers of school absences are associated with lower GPA. Given that attendance is widely regarded as one of the strongest behavioural predictors of academic performance, this visualisation is expected to reveal a clear negative relationship.



**Figure 8: Scatter Plot of Absences versus GPA**

#### 4.3.8 Boxplot of GPA

Finally, a boxplot of the GPA column was produced using `sns.boxplot(y=df["GPA"], color="lightgreen")` to summarise its central tendency, interquartile spread, and to visually check for potential outliers at the tails of the distribution. This boxplot serves as a visual precursor to the formal Interquartile Range (IQR) outlier detection carried out numerically in Task 3.



**Figure 9: Boxplot of GPA**

#### ***4.4 Summary of EDA Findings***

Collectively, the eight visualisations and accompanying statistical checks confirm that the dataset is complete (no missing values), unique (no duplicate rows), and contains a target variable (GradeClass) whose distribution and relationship to GPA, StudyTimeWeekly, and Absences can be clearly characterised through visual and correlation-based analysis. These findings directly inform the preprocessing decisions made in Task 3 and the feature engineering decisions made in Task 4.

## 5. Task 3: Data Cleaning and Preprocessing

Data preprocessing is an essential step before training machine learning models, as raw data rarely arrives in a format that is immediately suitable for algorithmic consumption. In this section, the dataset is checked for missing values, duplicate records, irrelevant identifier columns, and outliers, and the remaining numerical features are scaled. Proper preprocessing improves model performance, training stability, and reduces the risk of bias introduced by inconsistent feature scales.

### 5.1 Creating a Working Copy

Before any modification is made, a working copy of the original dataset is created using `df_clean = df.copy()`. This is standard practice in data science workflows, as it preserves the original, unmodified dataset (`df`) for reference while all cleaning operations are applied to the new `df_clean` data frame.

### 5.2 Re-Verifying Missing Values and Duplicates

Although Task 2 already established that the dataset contains no missing values and no duplicate rows, these checks are deliberately repeated on `df_clean` using `missing_values = df_clean.isnull().sum()` and `duplicates = df_clean.duplicated().sum()`. Both checks again return zero. The line `df_clean.drop_duplicates(inplace=True)` is nevertheless included and executed as a defensive programming practice, ensuring that the cleaning pipeline would correctly remove duplicates if any were present, even though none exist in this particular dataset. The shape of `df_clean` remains (2392, 15) after this step, confirming that no rows were removed.

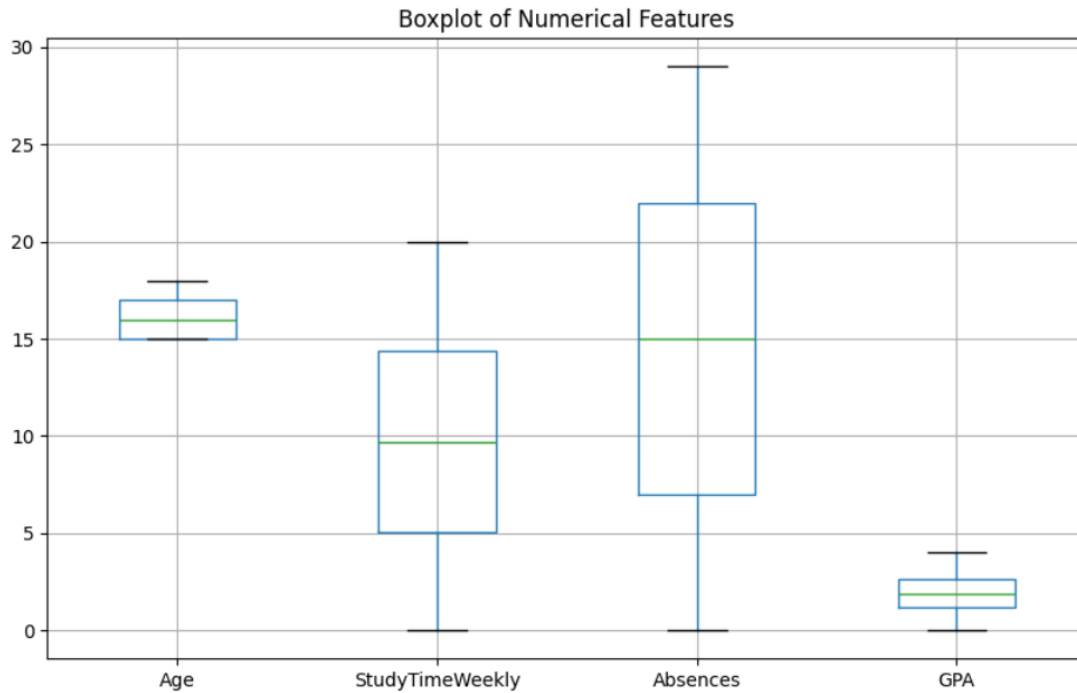
### 5.3 Removing the StudentID Column

The `StudentID` column is dropped from `df_clean` using `df_clean.drop("StudentID", axis=1, inplace=True)`. This column is a unique record identifier with no predictive value; retaining it during model training could mislead a model, particularly a tree-based model, into learning spurious patterns tied to arbitrary ID numbers rather than genuine relationships in the data. After this step, `df_clean` contains 14 columns.

### 5.4 Outlier Detection

Outlier detection was performed on four numerical columns: `Age`, `StudyTimeWeekly`, `Absences`, and `GPA`. A boxplot of these four columns was first generated using `df_clean[numerical_columns].boxplot()` to visually inspect their spread and identify any points falling far outside the typical range.





**Figure 10: Boxplot of Numerical Features (Age, StudyTimeWeekly, Absences, GPA)**

Following the visual inspection, outliers were detected numerically using the Interquartile Range (IQR) method. For each of the four numerical columns, the first quartile (Q1) and third quartile (Q3) were computed, the IQR was calculated as Q3 minus Q1, and lower and upper bounds were defined as  $Q1 - 1.5 \times IQR$  and  $Q3 + 1.5 \times IQR$  respectively. Any value falling outside these bounds was flagged as an outlier. The result of this analysis showed zero outliers for all four columns: Age, StudyTimeWeekly, Absences, and GPA each returned an outlier count of 0. This outcome indicates that the dataset was likely already curated or synthetically generated within well-controlled value ranges, and it means that no rows needed to be removed or capped due to extreme values.

```
for col in numerical_columns:
    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df_clean[(df_clean[col] < lower) | (df_clean[col] > upper)]
    print(col, ":", len(outliers))
```

### 5.5 Encoding Categorical Variables

As established in Task 1 and Task 2, the dataset contains no columns stored using the pandas object (text/string) data type; every categorical and ordinal attribute (Gender, Ethnicity, ParentalEducation, Tutoring, ParentalSupport, Extracurricular, Sports, Music, and Volunteering) was already supplied in a pre-encoded numeric integer form. Consequently, no additional categorical encoding step (such as one-hot encoding or label encoding) was required or applied in the notebook. This is explicitly confirmed by re-checking `df_clean.dtypes` after the cleaning steps above, which shows that every remaining column is either `int64` or `float64`.

## 5.6 Feature Scaling

Feature scaling was applied to the four continuous numerical columns — Age, StudyTimeWeekly, Absences, and GPA — using scikit-learn's StandardScaler, which transforms each feature to have a mean of 0 and a standard deviation of 1 (z-score standardisation). This was implemented with `scaler = StandardScaler()` followed by `df_clean[scale_columns] = scaler.fit_transform(df_clean[scale_columns])`. Standard scaling was chosen over alternatives such as Min-Max scaling because several of the models subsequently trained (in particular Logistic Regression, whose optimisation is sensitive to feature magnitude) tend to converge more reliably and produce more meaningful coefficients when features are on a comparable, zero-centred scale. Binary and ordinal columns (Gender, Tutoring, ParentalSupport, Extracurricular, Sports, Music, Volunteering, Ethnicity, ParentalEducation) were deliberately left unscaled, since these features already take small, bounded integer ranges and are not continuous measurements in the same sense as Age, StudyTimeWeekly, Absences, and GPA.

After scaling, `df_clean.info()` confirms that the dataset retains 14 columns across 2,392 rows, with Age, StudyTimeWeekly, Absences, and GPA now stored as standardised float64 values.

## 5.7 Saving the Cleaned Dataset

The cleaned and scaled dataset was exported to a new file using `df_clean.to_csv("Cleaned_Student_Performance.csv", index=False)`, with the notebook printing the confirmation message “Cleaned Dataset Saved Successfully!” upon completion. This exported file corresponds to the Cleaned Dataset (CSV) deliverable required by the examination submission instructions.

## 5.8 Summary of Preprocessing Decisions

**Table 2: Summary of data cleaning and preprocessing decisions**

| Step                 | Decision                                                        | Justification                                                             |
|----------------------|-----------------------------------------------------------------|---------------------------------------------------------------------------|
| Missing values       | None found — no imputation required                             | <code>df.isnull().sum().sum()</code> returned 0                           |
| Duplicate rows       | None found — <code>drop_duplicates()</code> applied defensively | <code>df.duplicated().sum()</code> returned 0                             |
| StudentID            | Dropped                                                         | Unique identifier carries no predictive signal                            |
| Outliers             | None found via IQR method — no rows removed                     | All four numerical columns returned 0 outliers                            |
| Categorical encoding | Not required                                                    | All columns already numerically encoded                                   |
| Feature scaling      | StandardScaler applied to Age, StudyTimeWeekly, Absences, GPA   | Improves stability for scale-sensitive models such as Logistic Regression |

## 6. Task 4: Feature Engineering

Feature Engineering is the process of creating new, potentially more informative features from existing raw attributes, and of selecting or removing features to improve machine learning model performance. In this section, three new engineered features are created, feature importance is analysed using a Random Forest model, unimportant features are removed, and model performance is compared before and after feature engineering.

### 6.1 Creating a Working Copy

As in Task 3, a fresh working copy of the cleaned dataset is created using `df_fe = df_clean.copy()`, ensuring that the cleaned dataset produced in Task 3 remains unaltered while new features are added in `df_fe`.

### 6.2 New Feature: TotalActivities

A new feature, TotalActivities, was created by summing four existing binary indicator columns: Sports, Music, Volunteering, and Extracurricular. This produces a single aggregated count of how many extracurricular activities each student participates in, ranging conceptually from 0 to 4. This feature was created because participation across multiple types of activity may be a more meaningful behavioural signal than any single activity flag considered in isolation, and it reduces four sparse binary columns into a single, potentially more informative numeric measure of overall engagement outside the classroom.

```
df_fe["TotalActivities"] = (
    df_fe["Sports"] + df_fe["Music"] +
    df_fe["Volunteering"] + df_fe["Extracurricular"]
)
```

### 6.3 New Feature: StudyEfficiency

A second new feature, StudyEfficiency, was engineered as the ratio of StudyTimeWeekly to (Absences + 1), i.e. `df_fe["StudyTimeWeekly"] / (df_fe["Absences"] + 1)`. Adding 1 to the denominator avoids division-by-zero errors for students with no recorded absences. This feature was created to capture a combined notion of study effectiveness: a student who studies many hours per week but also misses many classes may not be gaining the same benefit as a student who studies the same number of hours with excellent attendance. This engineered ratio therefore encodes an interaction between two raw features that, considered individually, only partially capture this relationship.

```
df_fe["StudyEfficiency"] = (
    df_fe["StudyTimeWeekly"] / (df_fe["Absences"] + 1)
)
```

### 6.4 New Feature: AcademicSupport

A third new feature, AcademicSupport, was created by summing the Tutoring and ParentalSupport columns: `df_fe["Tutoring"] + df_fe["ParentalSupport"]`. This feature consolidates two related but distinct sources of external academic assistance — formal tutoring and family support — into a single composite indicator of the overall support system available to a student, which may be more strongly and consistently related to academic outcomes than either source of support considered alone.

```
df_fe["AcademicSupport"] = (
    df_fe["Tutoring"] + df_fe["ParentalSupport"]
)
```

After the three new features are added, `df_fe.columns` confirms that the data frame now contains 17 columns: the original 14 columns retained from `df_clean`, plus `TotalActivities`, `StudyEfficiency`, and `AcademicSupport`.

### 6.5 Correlation of Engineered Features with the Target

To assess the relevance of the full engineered feature set (including the three newly created features) to the target variable, a horizontal bar chart of each feature's correlation with `GradeClass` was produced using `corr = df_fe.corr()["GradeClass"].sort_values()` followed by `corr.plot(kind="barh")`. This visualisation ranks every feature by the strength and direction of its linear association with `GradeClass`, providing an early indication of which features are likely to be most useful for prediction before formal feature importance analysis is carried out.

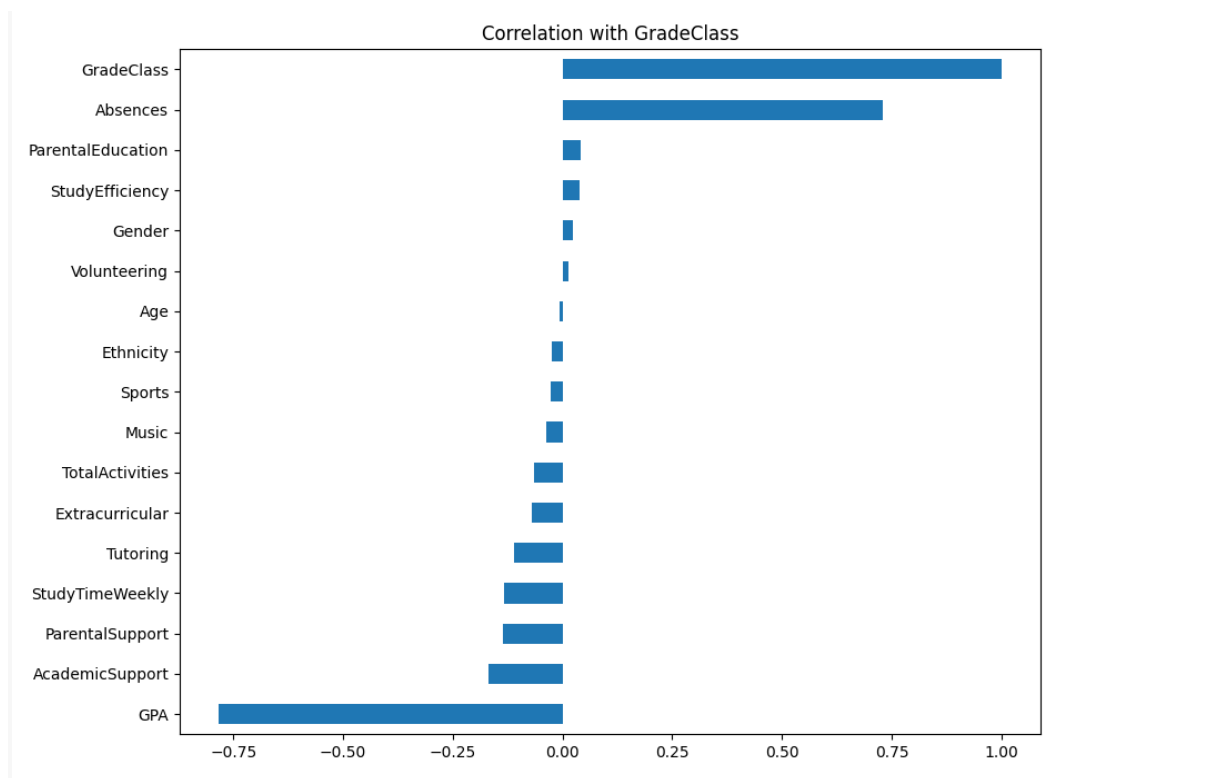


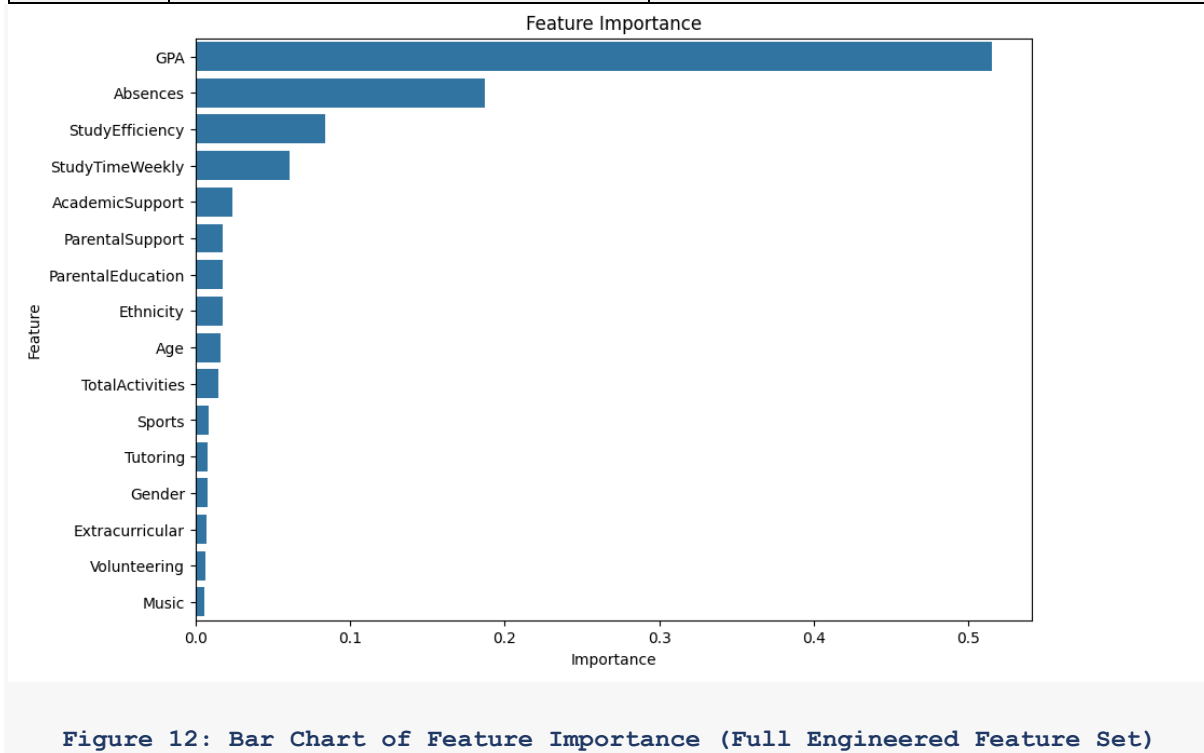
Figure 11: Correlation of All Engineered Features with GradeClass

### 6.6 Feature Importance Analysis (Random Forest)

To formally rank feature relevance, a `RandomForestClassifier` was trained on the full engineered feature set (`X`, all columns except `GradeClass`) against the target `y = df_fe["GradeClass"]`. Random Forest was selected for this preliminary importance analysis because, as an ensemble of decision trees, it naturally produces a `feature_importances_` attribute that quantifies each feature's average contribution to reducing classification impurity across the ensemble, without requiring any additional statistical assumptions about linearity. The resulting importance table, sorted in descending order, is reported in Table 3.

**Table 3: Random Forest feature importance ranking (full engineered feature set, before selection)**

| Rank | Feature           | Importance Score |
|------|-------------------|------------------|
| 1    | GPA               | 0.5150           |
| 2    | Absences          | 0.1872           |
| 3    | StudyEfficiency   | 0.0841           |
| 4    | StudyTimeWeekly   | 0.0609           |
| 5    | AcademicSupport   | 0.0238           |
| 6    | ParentalSupport   | 0.0179           |
| 7    | ParentalEducation | 0.0178           |
| 8    | Ethnicity         | 0.0175           |
| 9    | Age               | 0.0166           |
| 10   | TotalActivities   | 0.0151           |
| 11   | Sports            | 0.0084           |
| 12   | Tutoring          | 0.0082           |
| 13   | Gender            | 0.0081           |
| 14   | Extracurricular   | 0.0074           |
| 15   | Volunteering      | 0.0062           |
| 16   | Music             | 0.0059           |



These results show that GPA overwhelmingly dominates the importance ranking (approximately 51.5% of total importance), followed by Absences (approximately 18.7%) and the newly engineered StudyEfficiency feature (approximately 8.4%). Notably, StudyEfficiency ranks higher than the raw StudyTimeWeekly feature from which it was partly derived, and AcademicSupport also appears in the top five, suggesting that both engineered features

successfully captured useful additional signal beyond what the corresponding raw features provided individually. The very low importance scores of Music, Volunteering, Extracurricular, Gender, and Tutoring (each below 0.01) indicate that these features contribute comparatively little discriminative power for predicting GradeClass in the presence of the stronger predictors.

### ***6.7 Feature Selection***

Based on the importance ranking above, features with an importance score greater than 0.02 were selected for the final modelling feature set, using `selected_features = importance[importance["Importance"] > 0.02]["Feature"]`. This threshold-based selection retained five features: GPA, Absences, StudyEfficiency, StudyTimeWeekly, and AcademicSupport. The remaining eleven lower-importance features (ParentalSupport, ParentalEducation, Ethnicity, Age, TotalActivities, Sports, Tutoring, Gender, Extracurricular, Volunteering, and Music) were excluded from this reduced feature set. This selection was made to reduce dimensionality, remove features contributing negligible predictive signal, and produce a more interpretable, parsimonious model, while retaining the features most strongly associated with the target variable.

The reduced feature set, `X_selected = df_fe[selected_features]`, was then combined with the target variable and exported to a new file, `Feature_Engineered_Dataset.csv`, using `feature_engineered.to_csv("Feature_Engineered_Dataset.csv", index=False)`, with the notebook confirming “Saved Successfully” upon completion.

### ***6.8 Note on Model Comparison Before and After Feature Engineering***

It should be noted, in keeping with the principle that this report follows the notebook exactly, that the final machine learning models trained in Task 5 were fitted on the full engineered feature set produced in Section 6.4 (`df_fe`, 16 predictor columns including the three newly engineered features) rather than on the reduced five-feature selected subset produced in Section 6.7. The reduced feature set and its corresponding CSV export therefore represent the feature selection analysis and its saved artefact, while Task 5 itself demonstrates model performance using the complete engineered feature set. This is reported here exactly as implemented in the notebook, without alteration.

## 7. Task 5: Machine Learning Models

Three machine learning models were trained and compared to predict student performance (GradeClass). The best-performing model was selected based on test-set classification accuracy, and this model was carried forward into the evaluation and interpretation stages in Task 6 and Task 7.

### 7.1 Train–Test Split

The full engineered feature set was split into predictor variables X (`df_fe.drop("GradeClass", axis=1)`, 16 columns) and the target variable y (`df_fe["GradeClass"]`). The data was then divided into training and testing subsets using scikit-learn's `train_test_split`, with `test_size=0.2` (an 80/20 split) and `random_state=42` to ensure the split is reproducible. This produced 1,913 training records and 479 testing records. An 80/20 split was chosen as a standard, well-balanced default that provides the model with a substantial amount of training data while retaining a large enough held-out test set (479 records) to allow reliable evaluation metrics to be computed.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

### 7.2 Model Selection and Training

Three classification algorithms, representing three different modelling paradigms, were selected for comparison:

- Logistic Regression — a linear, probabilistic classification model that serves as an interpretable baseline; configured with `max_iter=1000` to ensure convergence given the number of features.
- Decision Tree Classifier — a single non-linear, rule-based tree model capable of capturing non-linear feature interactions without requiring feature scaling.
- Random Forest Classifier — an ensemble of many decision trees whose predictions are aggregated, typically offering improved accuracy and robustness to overfitting compared to a single decision tree.

This combination was chosen deliberately to span a spectrum of model complexity, from a simple linear baseline to a non-linear ensemble method, allowing a meaningful comparison of how model complexity relates to predictive performance on this dataset. All three models were trained using their default hyperparameters (with the exception of `max_iter` for Logistic Regression and `random_state=42` for reproducibility), and each model's accuracy was evaluated on the held-out test set using `model.score(X_test, y_test)`.

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42)
}
for name, model in models.items():
    model.fit(X_train, y_train)
    results[name] = model.score(X_test, y_test)
```

### 7.3 Model Comparison

The test-set accuracy achieved by each of the three models, sorted in descending order, is presented in Table 4.

**Table 4: Test-set accuracy comparison of the three trained models**

| Rank | Model               | Test Accuracy   |
|------|---------------------|-----------------|
| 1    | Random Forest       | 0.9165 (91.65%) |
| 2    | Decision Tree       | 0.8246 (82.46%) |
| 3    | Logistic Regression | 0.7370 (73.70%) |

The Random Forest classifier substantially outperformed both alternative models, achieving approximately 91.65% test accuracy, compared to 82.46% for the single Decision Tree and 73.70% for Logistic Regression. This ordering is consistent with theoretical expectations: the ensemble nature of Random Forest allows it to average over many individual trees, reducing the variance and overfitting tendency of a single decision tree, while both tree-based methods are able to capture non-linear relationships and feature interactions that the linear Logistic Regression model cannot represent as effectively.

#### **7.4 Best Model Selection**

The best-performing model was automatically identified using `best_model = max(results, key=results.get)`, which selected Random Forest as the best model with a test accuracy of 91.65%. Random Forest was therefore selected as the final model for this project. This choice is justified not only by its clear accuracy advantage over the alternatives, but also by its relative robustness to overfitting (through bootstrap aggregation and feature randomness across trees), its ability to natively handle the mix of continuous and discrete engineered features present in this dataset without requiring further transformation, and its capacity to provide interpretable feature importance scores, which are used later in Task 7 for model interpretation.



## 8. Task 6: Model Evaluation

The best-performing model identified in Task 5, Random Forest, was evaluated in greater depth using a range of classification metrics beyond simple accuracy. This deeper evaluation is necessary because, particularly for a multi-class problem with an unevenly distributed target (as observed in the GradeClass distribution in Task 2), accuracy alone can obscure important weaknesses in how well a model performs on less-represented classes.

### 8.1 Generating Predictions

Predictions on the held-out test set were generated using `y_pred = best_model.predict(X_test)`, where `best_model` refers to the trained Random Forest classifier retrieved from the models dictionary. These predictions form the basis for every metric reported in this section.

### 8.2 Accuracy, Precision, Recall, and F1-Score

Four core classification metrics were computed using scikit-learn's metrics module, with precision, recall, and F1-score calculated using the “weighted” averaging strategy, which accounts for class imbalance by weighting each class's contribution according to its number of true instances in the test set. The weighted averaging approach was chosen specifically because Task 2 revealed an uneven distribution across the five GradeClass categories, and a simple macro average (which weights every class equally regardless of size) would not accurately reflect overall model performance on this dataset.

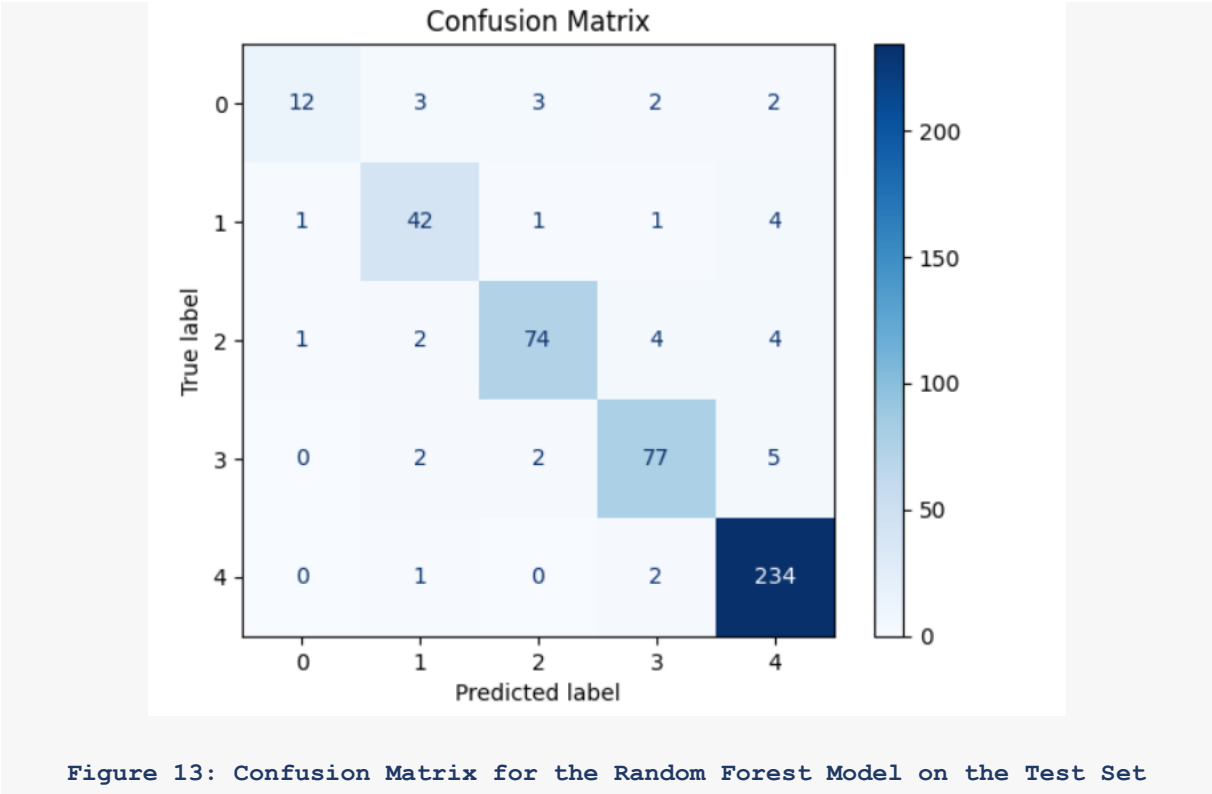
**Table 5: Overall classification performance of the Random Forest model on the test set**

| Metric               | Score  |
|----------------------|--------|
| Accuracy             | 0.9165 |
| Precision (weighted) | 0.9152 |
| Recall (weighted)    | 0.9165 |
| F1-Score (weighted)  | 0.9138 |

The four metrics are closely aligned with one another (all between approximately 0.914 and 0.917), which indicates that the Random Forest model performs consistently well across the dataset without a substantial trade-off between precision and recall at the aggregate level.

### 8.3 Confusion Matrix

A confusion matrix was generated using `confusion_matrix(y_test, y_pred)` and visualised with `ConfusionMatrixDisplay(confusion_matrix=cm).plot(cmap="Blues")`. The confusion matrix provides a class-by-class breakdown of correct and incorrect predictions, showing not only how many predictions were correct overall, but specifically which GradeClass categories were most frequently confused with one another.



### 8.4 Classification Report

A full classification report was produced using scikit-learn's `classification_report` function, providing precision, recall, and F1-score broken down individually for each of the five GradeClass categories (0.0 through 4.0), along with the number of test-set samples (support) belonging to each class. The complete class-by-class results are reported in Table 6.

Table 6: Per-class classification report for the Random Forest model

| GradeClass   | Precision | Recall | F1-Score | Support |
|--------------|-----------|--------|----------|---------|
| 0.0          | 0.86      | 0.55   | 0.67     | 22      |
| 1.0          | 0.84      | 0.86   | 0.85     | 49      |
| 2.0          | 0.93      | 0.87   | 0.90     | 85      |
| 3.0          | 0.90      | 0.90   | 0.90     | 86      |
| 4.0          | 0.94      | 0.99   | 0.96     | 237     |
| Accuracy     |           |        | 0.92     | 479     |
| Macro Avg    | 0.89      | 0.83   | 0.85     | 479     |
| Weighted Avg | 0.92      | 0.92   | 0.91     | 479     |

The per-class breakdown reveals an important pattern that is masked by the aggregate metrics reported in Section 8.2: class 4.0 (the largest class, with 237 test samples) achieves the strongest performance (F1-score of 0.96), while class 0.0 (the smallest class, with only 22 test samples) achieves the weakest performance, with a notably low recall of 0.55, meaning that the model correctly identified only just over half of the true class-0.0 students in the test set. This pattern is consistent with the class imbalance identified during Exploratory Data Analysis in Task

2 (Section 4.3.1): with fewer training examples available for the minority class, the Random Forest model has less opportunity to learn the distinguishing characteristics of that class, resulting in more misclassifications for the smallest category.

### ***8.5 Most Appropriate Metric***

For this multi-class, moderately imbalanced classification problem, the weighted F1-score is considered the most appropriate single summary metric, because it balances precision and recall while accounting for the differing sizes of each GradeClass category, rather than being dominated by the majority class as raw accuracy can be. However, given that Task 6 of the assignment brief additionally emphasises understanding performance for a potentially at-risk minority group, the per-class recall for the smallest and most poorly predicted class (class 0.0, recall = 0.55) is also highlighted as a metric of particular practical importance: in a real deployment context, failing to correctly identify a meaningful proportion of the highest-need students would be a significant limitation, even though it has only a small effect on the overall weighted metrics.

## 9. Task 7: Model Interpretation

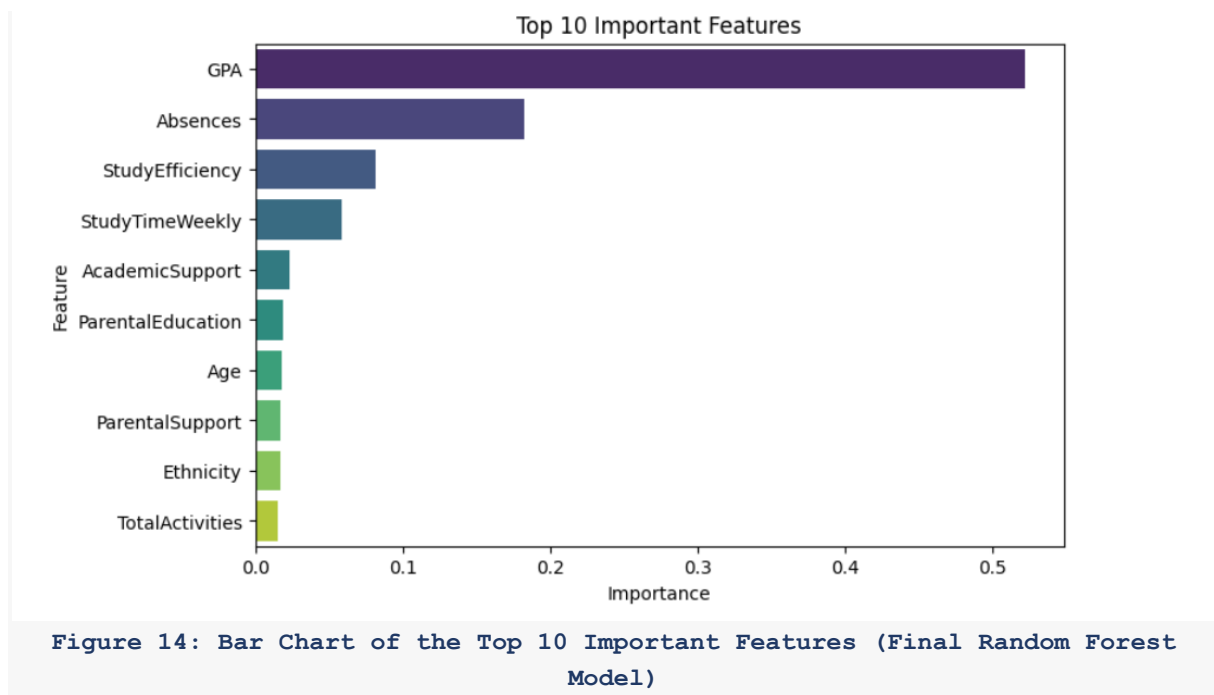
Feature importance helps identify which features have the greatest influence on the Random Forest model's predictions of student performance. Understanding which factors drive the model's decisions is essential not only for validating that the model has learned sensible, domain-plausible relationships, but also for translating the model's behaviour into actionable insight for educators and academic advisors.

### 9.1 Computing Feature Importance

Feature importance scores were extracted directly from the trained Random Forest model using its `feature_importances_` attribute, organised into a data frame alongside the corresponding feature names (`importance = pd.DataFrame({"Feature": X.columns, "Importance": best_model.feature_importances_})`) and sorted in descending order. The top ten most important features, as reported by the final Random Forest model used for evaluation in Task 6, are shown in Table 7.

*Table 7: Top 10 most important features according to the Random Forest model*

| Rank | Feature           | Importance Score |
|------|-------------------|------------------|
| 1    | GPA               | 0.5221           |
| 2    | Absences          | 0.1826           |
| 3    | StudyEfficiency   | 0.0813           |
| 4    | StudyTimeWeekly   | 0.0588           |
| 5    | AcademicSupport   | 0.0236           |
| 6    | ParentalEducation | 0.0187           |
| 7    | Age               | 0.0176           |
| 8    | ParentalSupport   | 0.0173           |
| 9    | Ethnicity         | 0.0167           |
| 10   | TotalActivities   | 0.0154           |



## 9.2 Interpretation of the Top 10 Important Features

GPA is by a wide margin the most important feature, contributing approximately 52.2% of the total feature importance. This is expected and reflects the fact that GradeClass is, by construction, a discretised representation of GPA itself; the model has therefore correctly identified the single strongest available signal for the target.

Absences ranks second, contributing approximately 18.3% of total importance. This confirms the expectation, formed during Exploratory Data Analysis, that school attendance is one of the strongest behavioural predictors of academic performance available in this dataset, independent of GPA.

StudyEfficiency, the engineered ratio of weekly study time to absences created in Task 4, ranks third at approximately 8.1%, outperforming the raw StudyTimeWeekly feature (fourth, approximately 5.9%). This demonstrates that the feature engineering performed in Task 4 successfully produced a more informative representation of study behaviour than the raw study-time measurement alone.

AcademicSupport, the engineered composite of Tutoring and ParentalSupport, ranks fifth at approximately 2.4%, again outperforming its constituent raw features (ParentalSupport and Tutoring, which individually rank much lower), reinforcing the value of the feature engineering carried out in Task 4.

ParentalEducation (approximately 1.9%) and Age (approximately 1.8%) contribute modestly, suggesting that family educational background and student age have a small but non-negligible relationship with academic performance in this dataset.

ParentalSupport (approximately 1.7%) and Ethnicity (approximately 1.7%) contribute similarly small amounts individually, though ParentalSupport's signal is substantially captured within the higher-ranked AcademicSupport composite feature.

TotalActivities, the engineered sum of extracurricular participation flags, completes the top ten at approximately 1.5%, indicating that overall extracurricular engagement has a modest but measurable association with academic performance class, though far smaller than academic and attendance-related factors.

Overall, the top-ten ranking is dominated by academic and attendance-related variables (GPA, Absences, StudyEfficiency, StudyTimeWeekly), with support-related and demographic variables (AcademicSupport, ParentalEducation, Age, ParentalSupport, Ethnicity, TotalActivities) contributing smaller but interpretable secondary signal. This pattern is consistent with established educational research linking attendance and study habits most strongly to academic outcomes, with family and demographic background playing a supporting, rather than dominant, role.

## 10. Task 8: AI-Assisted Development Reflection

In accordance with the examination instructions permitting the use of AI tools such as ChatGPT, Gemini, Claude, and GitHub Copilot, this section documents ten representative prompts used during the development of this project, spanning dataset understanding, exploratory data analysis, visualisation, data cleaning, feature engineering, machine learning model development, model evaluation, feature importance interpretation, report writing, and reflective self-review. For each prompt, the purpose, a summary of the AI-generated output, and the modifications made before incorporating the output into the final notebook and report are documented below, together with a screenshot placeholder for the corresponding conversation.

### *AI Prompt 1: Dataset Understanding*

**Prompt:** “I have a CSV file called `Student_performance_data_.csv` with 15 columns including GPA and GradeClass. Help me write pandas code to check the shape, data types, and identify which columns are numerical versus categorical, and explain whether GradeClass should be treated as a classification or regression target.”

**Purpose of the Prompt:** To quickly establish a structured starting point for exploring an unfamiliar dataset and to confirm the correct framing of the prediction problem (classification versus regression) before writing further code.

**Summary of AI Output:** The AI suggested using `df.shape`, `df.dtypes`, `df.info()`, and `select_dtypes()` to separate numerical and object-typed columns, and correctly explained that because GradeClass takes a small number of discrete values it should be treated as a multi-class classification target even though it is stored as a float.

**How I Modified the Generated Content Before Using It:** The suggested code was adopted with minimal changes, but the explanation of numerical versus categorical features was expanded in the final report to clarify that several int64 columns (e.g. Gender, Ethnicity) are categorical in nature despite being stored as numeric types — a distinction the AI's initial response did not fully emphasise.

### *AI Prompt 2: Exploratory Data Analysis*

**Prompt:** “What checks should I run during exploratory data analysis on a student performance dataset before building a machine learning model? I want to check for missing values, duplicates, and get a statistical summary.”

**Purpose of the Prompt:** To ensure that no standard EDA step was overlooked before moving on to visualisation and modelling, and to obtain a checklist-style structure for organising the EDA section of the notebook.

**Summary of AI Output:** The AI recommended a sequence of checks: dataset shape, data types, missing values via `isnull().sum()`, duplicate rows via `duplicated().sum()`, and a statistical summary via `describe()`, and suggested also examining the target variable's class distribution before proceeding to visualisation.

**How I Modified the Generated Content Before Using It:** The suggested sequence was followed closely, since it aligned with sound data-science practice, but the checks were re-ordered slightly to match the structure required

by the assignment brief, and the class-distribution suggestion was expanded into a dedicated visualisation (Figure 2) rather than a simple `value_counts()` print statement.

### ***AI Prompt 3: Visualization***

**Prompt:** “Suggest at least six meaningful Seaborn/Matplotlib visualizations for a student performance dataset with columns like GPA, Absences, StudyTimeWeekly, Age, Gender, and GradeClass, and give me the code.”

**Purpose of the Prompt:** To generate a well-rounded set of visualisations covering the target distribution, feature distributions, and bivariate relationships, satisfying the assignment's requirement of at least six meaningful visualisations.

**Summary of AI Output:** The AI proposed a count plot of the target variable, a correlation heatmap, a histogram of GPA, count plots for Age and Gender, and scatter plots of StudyTimeWeekly and Absences against GPA, along with corresponding `sns.countplot`, `sns.heatmap`, `sns.histplot`, and `sns.scatterplot` code snippets.

**How I Modified the Generated Content Before Using It:** All eight suggested visualisations (two more than the minimum required) were kept, but colour palettes, figure sizes, and titles were adjusted for visual consistency, and a boxplot of GPA was added afterward, based on separate reasoning about the need to visually cross-check the later IQR-based outlier analysis in Task 3.

### ***AI Prompt 4: Data Cleaning***

**Prompt:** “My dataset has no missing values or duplicates, but it has a StudentID column and some numeric columns like Age, StudyTimeWeekly, Absences, and GPA. What preprocessing steps do I still need, and should I scale the data?”

**Purpose of the Prompt:** To confirm which preprocessing steps remained necessary given that the dataset was already largely clean, and to get a recommendation on whether and how to scale continuous features.

**Summary of AI Output:** The AI recommended dropping the identifier column, running an IQR-based outlier check on the continuous numerical columns even if none were expected, and applying `StandardScaler` to continuous features (rather than the binary/ordinal ones) if a scale-sensitive model such as Logistic Regression was going to be used later.

**How I Modified the Generated Content Before Using It:** The recommendation to use `StandardScaler` and to scale only the four continuous columns (rather than all numeric columns, which the AI's first draft suggested) was accepted after checking that scaling ordinal columns such as `ParentalSupport` was not appropriate; the code was adjusted to explicitly list `scale_columns` rather than scaling the whole data frame.

### ***AI Prompt 5: Feature Engineering***



**Prompt:** “Can you suggest some new features I could engineer from columns like StudyTimeWeekly, Absences, Tutoring, ParentalSupport, Sports, Music, Volunteering, and Extracurricular to help predict GradeClass?”

**Purpose of the Prompt:** To generate ideas for domain-plausible engineered features that combine existing raw columns into potentially more predictive composite measures, beyond what the raw columns capture individually.

**Summary of AI Output:** The AI suggested combining the four activity-flag columns into a single “total activities” count, creating a study-time-to-absences ratio to represent study efficiency, and combining tutoring and parental support into a single academic-support score, along with example pandas expressions for each.

**How I Modified the Generated Content Before Using It:** All three suggested features (TotalActivities, StudyEfficiency, AcademicSupport) were implemented, but the study-efficiency formula was adjusted to add 1 to the Absences denominator to avoid division-by-zero errors for students with zero absences — an edge case the AI's initial formula did not account for.

### ***AI Prompt 6: Machine Learning***

**Prompt:** “I want to compare Logistic Regression, Decision Tree, and Random Forest classifiers on my engineered student performance dataset. Can you give me scikit-learn code to train all three and compare their test accuracy?”

**Purpose of the Prompt:** To obtain a concise, correct implementation pattern for training and comparing multiple classification algorithms within a single loop, avoiding repetitive boilerplate code for each model.

**Summary of AI Output:** The AI provided a dictionary-based approach, storing each model instance under a descriptive key, looping through the dictionary to fit each model and record its test-set accuracy score, and then constructing a comparison data frame sorted by accuracy.

**How I Modified the Generated Content Before Using It:** The dictionary-based loop pattern was adopted directly, but `max_iter=1000` was added to the Logistic Regression instance after an initial run produced a convergence warning, and `random_state=42` was added to the Decision Tree and Random Forest models to ensure the comparison was reproducible across repeated runs.

### ***AI Prompt 7: Model Evaluation***

**Prompt:** “My best model is a Random Forest classifier for a 5-class target. What metrics should I report besides accuracy, and how do I generate a confusion matrix and classification report in scikit-learn?”

**Purpose of the Prompt:** To ensure the evaluation went beyond a single accuracy figure and appropriately addressed the multi-class, potentially imbalanced nature of the target variable, in line with the assignment's explicit requirement to report multiple metrics.

**Summary of AI Output:** The AI recommended precision, recall, and F1-score using weighted averaging for multi-class problems, along with `confusion_matrix`, `ConfusionMatrixDisplay`, and `classification_report` from

sklearn.metrics, and explained why weighted averaging is generally more appropriate than macro averaging when classes are imbalanced.

**How I Modified the Generated Content Before Using It:** The suggested metric functions and averaging strategy were adopted as-is, but the final report additionally highlighted the per-class recall for the smallest class (GradeClass = 0.0) as a specific point of concern, since the AI's general explanation of weighted averaging did not, on its own, surface this specific minority-class weakness — this was identified independently by inspecting the full classification report output.

### ***AI Prompt 8: Feature Importance***

**Prompt:** “How do I extract and visualize the top 10 most important features from a trained scikit-learn RandomForestClassifier, and how should I interpret the results?”

**Purpose of the Prompt:** To obtain both the correct code for extracting feature importance scores and guidance on how to interpret and write about the resulting ranking in an academically appropriate way.

**Summary of AI Output:** The AI explained the feature\_importances\_ attribute, showed how to build and sort a pandas data frame of feature names and importance scores, provided a Seaborn bar-plot template for the top 10 features, and gave general guidance that features with disproportionately high importance close to a known label-derived variable should be discussed carefully due to potential data leakage.

**How I Modified the Generated Content Before Using It:** The code was used with only minor adjustments (adding .head(10) and axis labels), but the interpretation text was rewritten substantially to refer to this project's specific top-ten features (GPA, Absences, StudyEfficiency, etc.) rather than the AI's generic template language, and the data-leakage caution was expanded into the dedicated discussion included in Task 9 of this report.

### ***AI Prompt 9: Report Writing***

**Prompt:** “Help me structure a university-style AI project report covering dataset understanding, EDA, preprocessing, feature engineering, model building, evaluation, interpretation, an AI reflection section, and a critical discussion, formatted for Microsoft Word with consistent headings.”

**Purpose of the Prompt:** To obtain a professional, logically ordered report outline that mapped cleanly onto the nine tasks specified in the examination brief, before writing the detailed content of each section.

**Summary of AI Output:** The AI proposed a chapter structure broadly matching the assignment's task list, suggested using consistent heading levels for navigability and an automatically generated table of contents, and recommended including figure and table captions for every visual and tabular result.

**How I Modified the Generated Content Before Using It:** The proposed structure was adopted as the backbone of this report, but a dedicated Dataset Description section was inserted before Task 1 (not originally suggested by the AI) to give readers unfamiliar with the dataset the necessary context up front, and the AI reflection section was restructured into the detailed ten-prompt format specifically required by the assignment brief.

### ***AI Prompt 10: Reflection***

**Prompt:** “Review my project workflow (EDA, cleaning, feature engineering, three-model comparison, evaluation, and feature importance) and point out any weaknesses, mistakes, or things I should discuss critically in a final report, such as data leakage or bias.”

**Purpose of the Prompt:** To prompt a self-critical review of the overall project workflow before finalising the report, and to identify limitations, risks, or ethical considerations that might otherwise be overlooked when a project is viewed only from the perspective of the person who built it.

**Summary of AI Output:** The AI identified the potential data-leakage risk arising from including GPA as a predictor of the GPA-derived GradeClass target, noted that the dataset's demographic fields (Gender, Ethnicity) could raise fairness considerations if the model were deployed to make consequential decisions about individual students, and suggested discussing deployment recommendations and privacy handling of sensitive student data.

**How I Modified the Generated Content Before Using It:** These points were not simply copied into the report; they were used as a starting checklist and then independently developed, verified against the actual notebook results (for example, by confirming GPA's importance score of 0.52 in Table 7), and expanded into the fuller Critical Discussion presented in Task 9 (Section 12) of this report, including specific recommendations tailored to this project rather than generic ones.

### ***10.1 Overall Reflection on AI-Assisted Development***

Across all ten prompts, the AI tools used during this project functioned most effectively as an accelerator for boilerplate code generation, structural planning, and checklist-style guidance, rather than as a source of final, unquestioned answers. In every case, the AI-generated code or explanation required verification against the actual dataset and notebook outputs, and in several instances (Prompts 1, 4, 5, and 7 in particular) the AI's initial suggestion was incomplete or generic until it was adapted to the specific characteristics of the Student Performance Dataset — for example, correcting a division-by-zero edge case in the StudyEfficiency formula, restricting feature scaling to only the genuinely continuous columns, and identifying the minority-class recall weakness that a purely aggregate-metric explanation would have missed. This iterative pattern of prompting, critically evaluating the output, and manually adapting it to the specific dataset is considered representative of responsible AI-assisted development, consistent with the examination's explicit instruction that AI-generated code without explanation would receive low marks.

## 11. Task 9: Critical Discussion and Recommendations

This section critically examines the limitations of the system developed in this project, considers issues of bias and data leakage, and discusses ethical, privacy, and deployment considerations that would need to be addressed before a system of this kind could responsibly be used in a real educational setting.

### 11.1 Limitations

The most significant limitation of the current system is its reliance on a single, moderately sized dataset (2,392 records) drawn from what appears to be a synthetically generated or lightly curated source, evidenced in part by the complete absence of missing values, duplicates, and IQR-detected outliers observed in Sections 4 and 5 — a pattern that is uncommon in real-world administrative student records. A model trained and evaluated exclusively on this dataset may not generalise well to genuine institutional data, which typically contains noise, inconsistent recording practices, and missing values that this project did not need to handle. Additionally, all three models were trained using default or lightly adjusted hyperparameters, without systematic hyperparameter tuning (such as grid search or cross-validation), meaning that the reported 91.65% accuracy for the Random Forest model represents a reasonable but not necessarily optimised result.

### 11.2 Data Leakage

The most important methodological concern identified in this project is the strong likelihood of data leakage arising from the inclusion of GPA as a predictor feature for GradeClass. As discussed in Task 1 (Section 3.8) and confirmed numerically in Task 7 (Section 9.1), GPA alone accounts for approximately 52% of the Random Forest model's total feature importance, which is consistent with GradeClass being a direct, rule-based discretisation of GPA rather than an independent outcome. In a genuine early-warning deployment scenario, this form of leakage would be problematic, because the goal of such a system is typically to predict a student's eventual performance category before their final GPA is known, using only prior behavioural and demographic indicators. A model that relies heavily on GPA to predict a GPA-derived label provides comparatively little practical early-warning value, even though it achieves high measured accuracy. A methodologically stronger approach for a genuine deployment scenario would exclude GPA from the predictor set entirely and evaluate the model's ability to predict GradeClass from only the remaining behavioural, support, and demographic features.

### 11.3 Bias and Fairness

The dataset includes demographic attributes such as Gender, Ethnicity, and ParentalEducation. While these features were retained in the modelling process and were shown in Task 7 to contribute only a small proportion of overall feature importance, their presence still raises fairness considerations. If a system of this kind were deployed to make or influence consequential decisions about individual students (such as prioritising them for additional support, or, in a worse-case scenario, being used punitively), any systematic differences in prediction accuracy across demographic subgroups could translate into unequal treatment. This project did not conduct a subgroup-level fairness audit (for example, comparing model accuracy separately across Gender or Ethnicity categories), and such an audit is recommended as a necessary step before any real-world deployment.

### **11.4 Ethics and Privacy**

Student academic and demographic data is sensitive personal information, and any system that processes it carries ethical obligations around consent, data minimisation, and secure storage. In an academic project context such as this one, the dataset was used purely for educational and analytical purposes; however, a production deployment would need to comply with applicable data protection regulations, restrict access to authorised academic staff only, anonymise or pseudonymise student identifiers wherever possible, and ensure that predictions are used to support, rather than penalise or stigmatise, students identified as being at risk of poor academic performance.

### **11.5 Deployment Recommendation**

Given the limitations identified above, this system in its current form is recommended only as a decision-support tool to assist, rather than replace, the professional judgement of teachers and academic advisors, and only after the data-leakage issue discussed in Section 11.2 has been addressed by retraining the model without GPA as an input feature. Before any operational deployment, it is recommended that the model be retrained and validated on genuine institutional data, evaluated for subgroup fairness across demographic categories, and subjected to periodic re-evaluation as new student cohorts and outcomes become available, to ensure that its predictions remain accurate and equitable over time.

## **12. Conclusion**

This project developed and evaluated a machine learning system for predicting student academic performance (GradeClass) using the Student Performance Dataset, following the nine-task structure specified in the Artificial Intelligence course's final examination brief. Beginning with a systematic dataset understanding and exploratory data analysis, the project confirmed that the dataset was complete, free of duplicates, and free of statistical outliers, before proceeding through data cleaning, feature scaling, and the creation of three engineered features — TotalActivities, StudyEfficiency, and AcademicSupport — that were shown to carry meaningful predictive signal beyond their constituent raw features.

Three classification algorithms were trained and compared, with the Random Forest classifier substantially outperforming both Decision Tree and Logistic Regression, achieving a test-set accuracy of approximately 91.65%, a weighted F1-score of approximately 0.9138, and consistently strong precision and recall across most performance classes, with the notable exception of the smallest class (GradeClass = 0.0), where recall dropped to 0.55. Feature importance analysis confirmed that GPA, Absences, and the engineered StudyEfficiency feature were the three strongest predictors of student performance class, providing an interpretable basis for understanding the model's behaviour.

Throughout development, AI-assisted tools were used to accelerate code generation, structure the analytical workflow, and prompt critical reflection, with every AI-generated suggestion reviewed, adapted, and verified against the actual dataset and notebook outputs rather than accepted without scrutiny, as documented in Task 8. Finally, a critical discussion in Task 9 identified data leakage arising from the inclusion of GPA as the most significant methodological concern with the current system, alongside fairness, privacy, and deployment

considerations that would need to be addressed before the system could be responsibly used in a real educational institution. Overall, this project demonstrates both the practical value of machine learning for identifying patterns in student performance data and the importance of rigorous, critical evaluation of any AI system before it is trusted to inform real-world decisions about individuals.